

Terraform 3 Task

Why Terraform is important

1. **Infrastructure as Code (IaC)**
Write infrastructure in .tf files → version-controlled, repeatable, and auditable.
2. **Automation & Speed**
Create servers, networks, databases, and services in minutes with init → plan → apply.
3. **Consistency & No Human Error**
Same code = same infrastructure across dev, test, and prod.
4. **Multi-Cloud Support**
Works with AWS, Azure, GCP, and on-prem from a **single tool**.
5. **State Management**
Tracks real infrastructure using a **state file**, enabling safe updates and drift detection.
6. **Idempotency**
Running Terraform multiple times won't break infra — it only changes what's needed.
7. **Easy CI/CD Integration**
Integrates smoothly with Jenkins, GitHub Actions, GitLab CI, etc.
8. **Scalable & Modular**
Reuse infrastructure using **modules** as your environment grows.
9. **Cost & Resource Control**
Plan before apply → know exactly what will be created or destroyed.
10. **Industry Standard**
Widely used in DevOps and cloud roles; strong interview and production relevance.

One-line interview answer

Terraform is important because it automates infrastructure provisioning using code, ensuring consistency, scalability, and multi-cloud support with safe, repeatable deployments.

Different Types variables:

Using of variables:

=====

1) By using variables.tf file

2) By using interactive mode (This will get activated if we don't pass default value in variable.tf file)

3) Command line flags

--> terraform apply -var "filename=/root/pets.txt" -var "prefix=MR"

4) Environment variables

--> export TF_VAR_filename="/root.pets.txt"

--> export TF_VAR_prefix= "MR"

--> Set-Item -Path env:TF_VAR_filename -Value 'wild.txt'

terraform apply

5) variable definition file (Should be end with terraform.tfvars/terraform.tfvars.json)

--> for automatically loaded file name *.auto.tfvars/*.auto.tfvars.json

--> if we are saving the file with other name like variable.tfvars then we need to pass this in CLI

--> terraform apply -var-file variable.tfvars

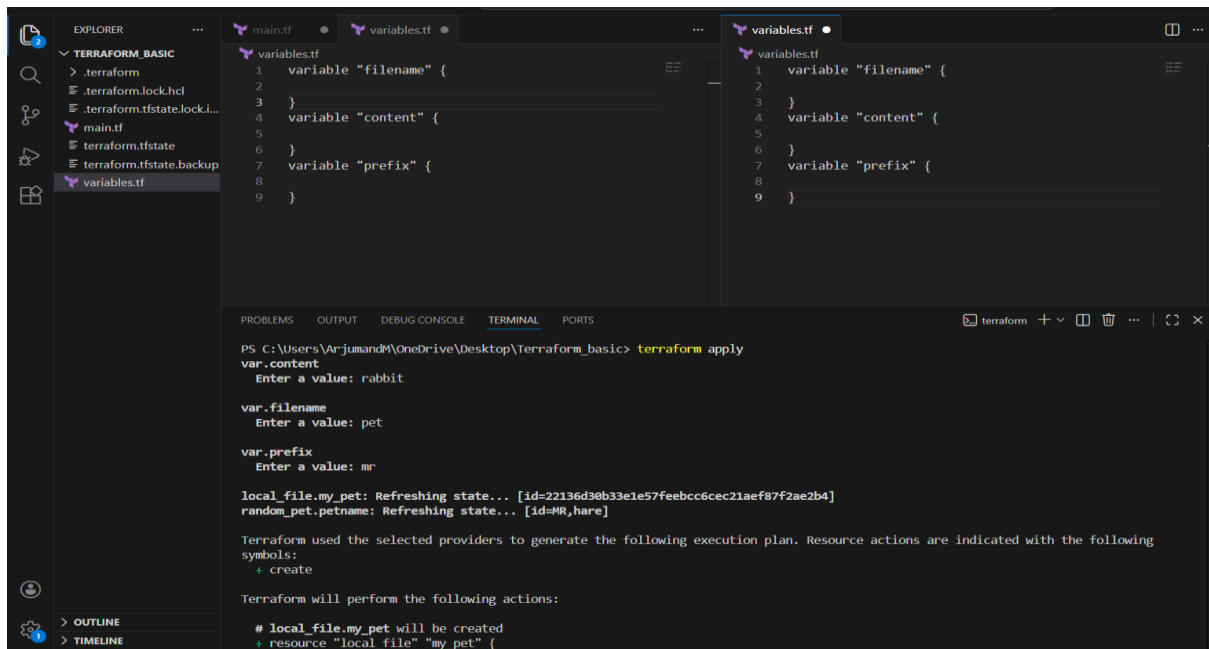
Execute the script shown in the Terraform video 03

1. By using interactive mode (This will get activated if we don't pass default value in variable.tf file)

--- > Go to visual studio and open main.tf file and now create variable.tf and then don't pass the values in it/

--- > after applying the terraform apply it will ask values enter the values

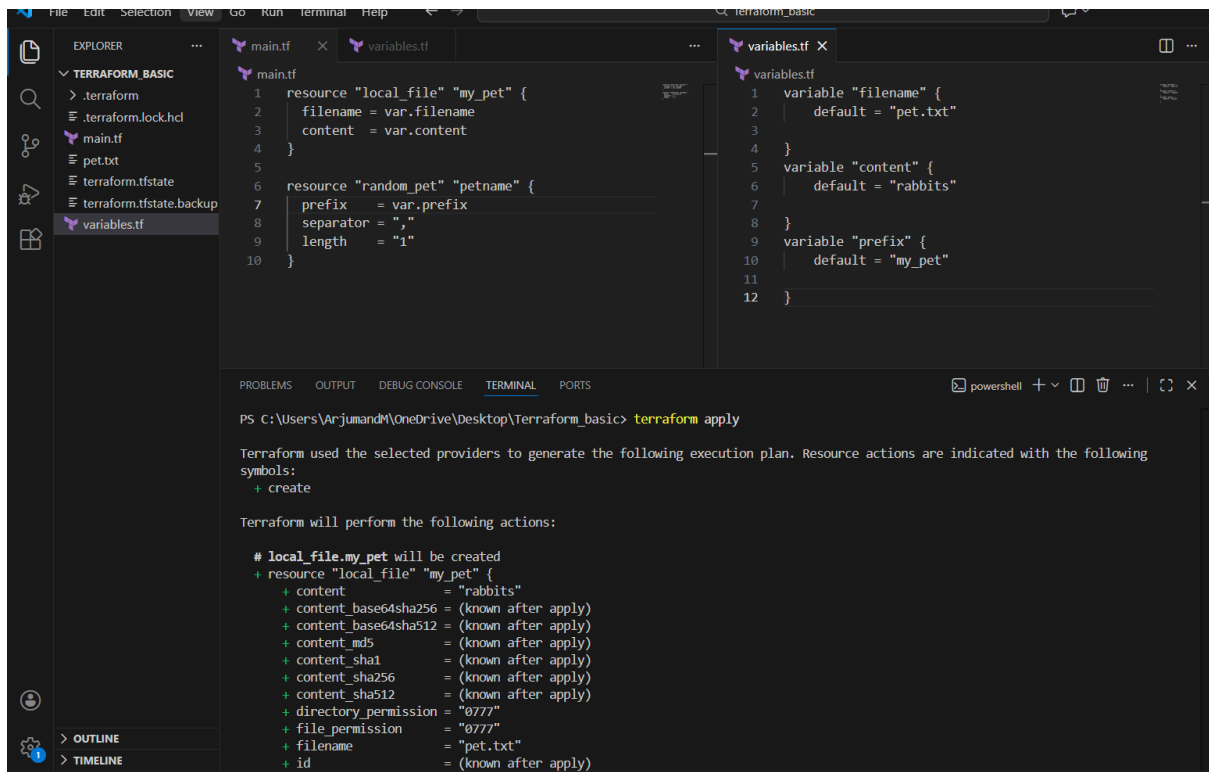
--- > This type is called as interactive mode.



2. By using variables.tf file (Input or default variable)

--- > we have to define variable in variable.tf file itself so that it will take the values

--- > After entering the values, we must enter terraform apply to after saving the values



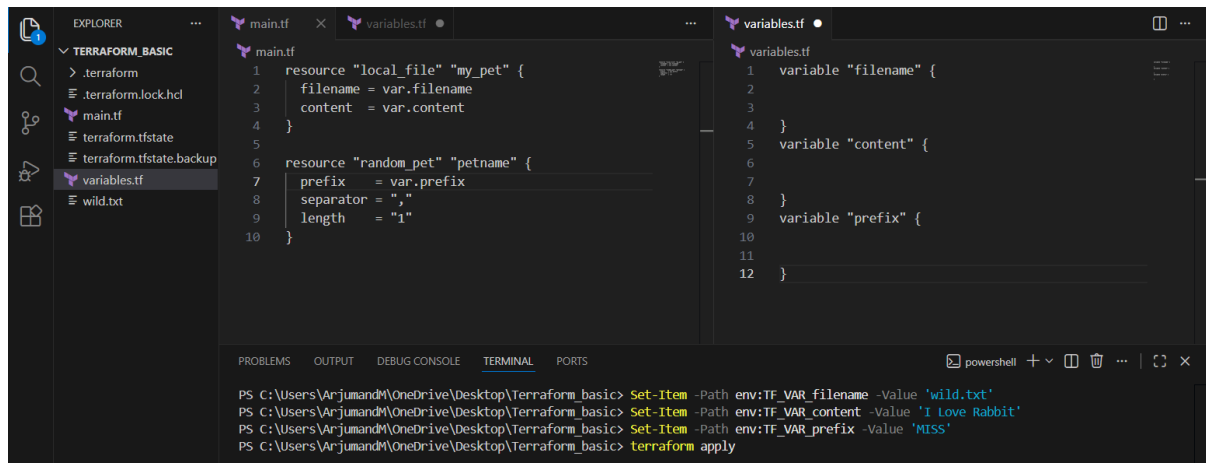
3. Environment variables (TF var)

---> We have to provide values on using TF_VAR as shown in the image

Set-Item -path env:TF_VAR_filename -value "wild.txt"

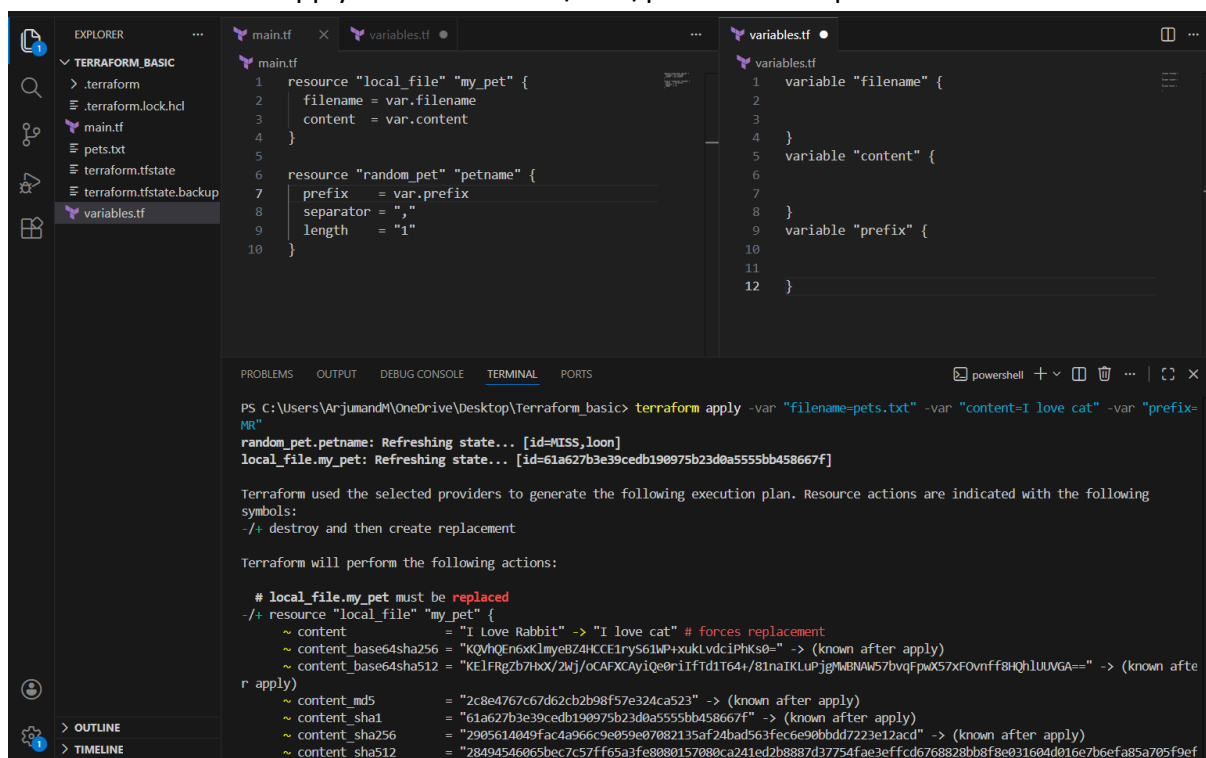
Set-Item -path env:TF_VAR_content -value "I love rabbit"

Set-Item -path env:TF_VAR_prefix -value "MISS"



4. Command line flags:

➤ --> terraform apply -var "filename=/root/pets.txt" -var "prefix=MR"

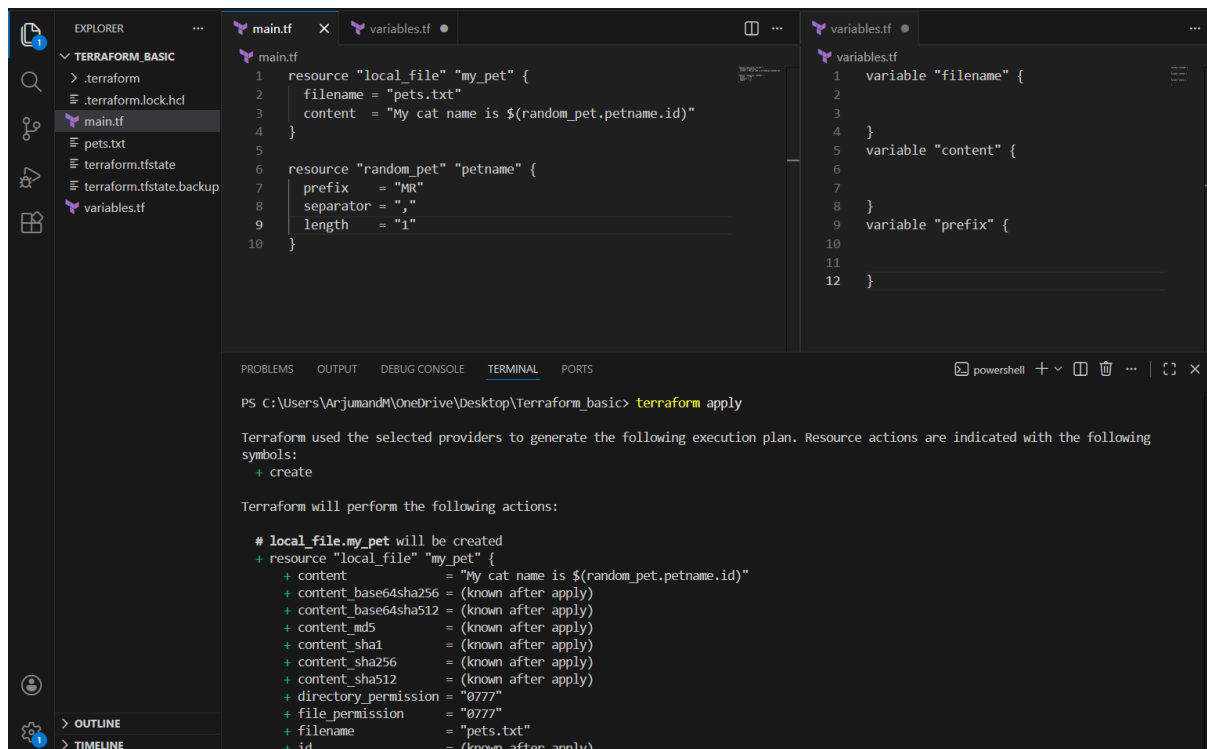


5) varibale definition file (Should be end with terraform.tfvars/terraform.tfvars.json)

--> for automatically loaded file name *.auto.tfvars/*.auto.tfvars.json

--> if we are saving the file with other name like variable.tfvars then we need to pass this in CLI

--> terraform apply -var-file varibale.tfvars



The screenshot shows a VS Code editor with two files open: `main.tf` and `variables.tf`. The `main.tf` file contains a `resource "local_file" "my_pet"` and a `resource "random_pet" "petname"`. The `variables.tf` file contains variables for `filename`, `content`, and `prefix`. The terminal window shows the command `terraform apply` being executed, resulting in the creation of the `local_file.my_pet` resource.

```
main.tf
1 resource "local_file" "my_pet" {
2   filename = "pets.txt"
3   content = "My cat name is ${random_pet.petname.id}"
4 }
5
6 resource "random_pet" "petname" {
7   prefix = "MR"
8   separator = ","
9   length = "1"
10 }
```

```
variables.tf
1 variable "filename" {
2
3 }
4
5 variable "content" {
6
7 }
8
9 variable "prefix" {
10
11 }
12 }
```

```
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic> terraform apply

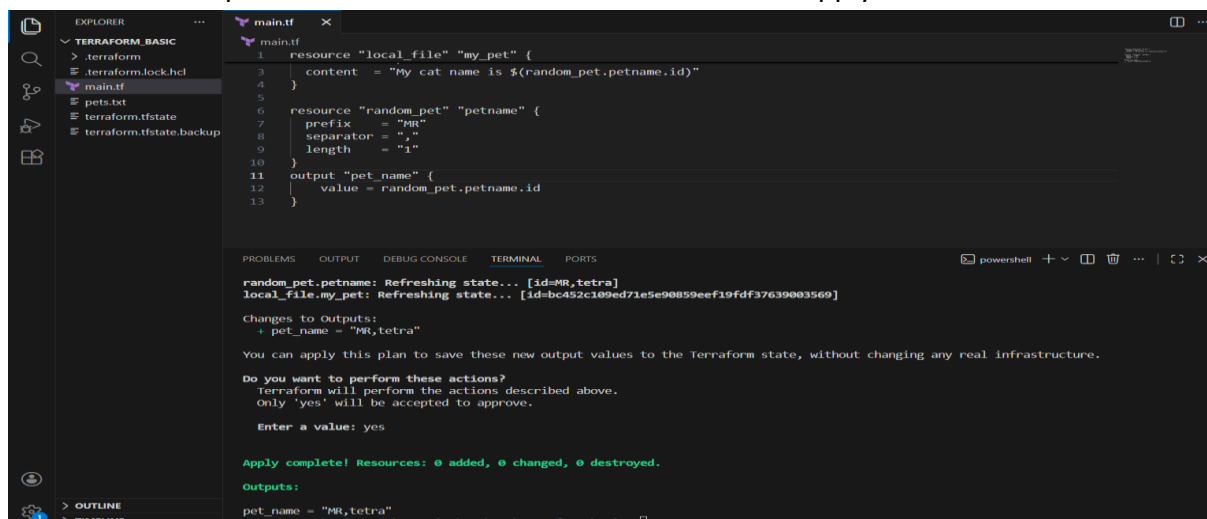
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  + create

Terraform will perform the following actions:

# local_file.my_pet will be created
+ resource "local_file" "my_pet" {
  + content = "My cat name is ${random_pet.petname.id}"
  + content_base64sha256 = (known after apply)
  + content_base64sha512 = (known after apply)
  + content_md5 = (known after apply)
  + content_sha1 = (known after apply)
  + content_sha256 = (known after apply)
  + content_sha512 = (known after apply)
  + directory_permission = "0777"
  + file_permission = "0777"
  + filename = "pets.txt"
  + id = (known after apply)
```

OUTPUT VARIABLE:

➤ Addb output variable in main.tf then enter terraform apply



The screenshot shows the same VS Code editor with the `main.tf` file updated to include an `output "pet_name"` block. The terminal window shows the command `terraform apply` being executed, resulting in the creation of the `pet_name` output variable.

```
main.tf
1 resource "local_file" "my_pet" {
2   content = "My cat name is ${random_pet.petname.id}"
3 }
4
5
6 resource "random_pet" "petname" {
7   prefix = "MR"
8   separator = ","
9   length = "1"
10 }
11 output "pet_name" {
12   value = random_pet.petname.id
13 }
```

```
random_pet.petname: Refreshing state... [id=MR,tetra]
local_file.my_pet: Refreshing state... [id=bc452c109ed71e5e90859eef19fdf37639003569]

Changes to Outputs:
  + pet_name = "MR,tetra"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
pet_name = "MR,tetra"
PS C:\Users\ArjunandM\OneDrive\Desktop\Terraform_basic>
```

- Random name is provided and displayed the output as well.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
pet_name = "MR,tetra"
```

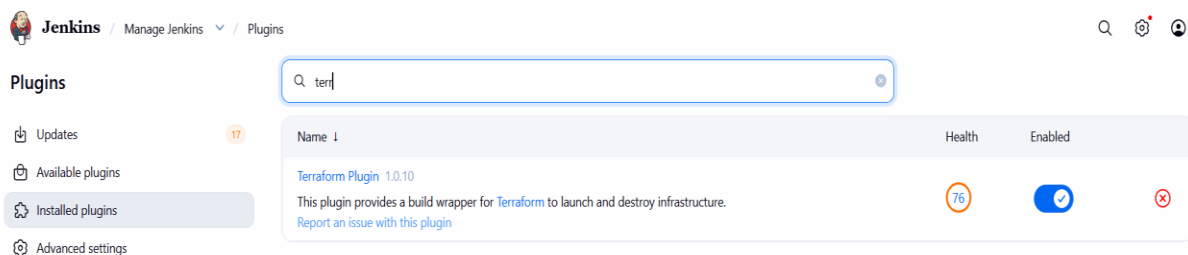
```
PS C:\Users\ArjumandM\OneDrive\Desktop\Terraform_basic> |
```

Integrate Terraform in Jenkins using the Terraform plugin.

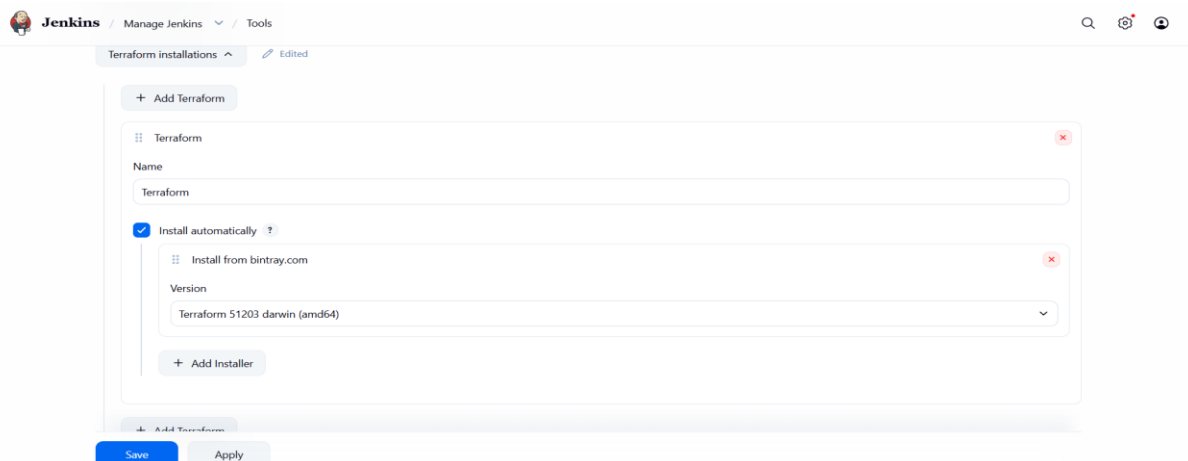
- Install terraform in CLI from official terraform site
- After installing terraform check the version for make sure

```
[root@ip-172-31-21-117 ~]# terraform -version
Terraform v1.14.3
on linux_amd64
[root@ip-172-31-21-117 ~]# |
```

- Install Terraform plugins, go to manage Jenkins and plugins
- Search for terraform plugin



- After installing plugins add plugin in global tools
- Go to terraform section and provide name and version

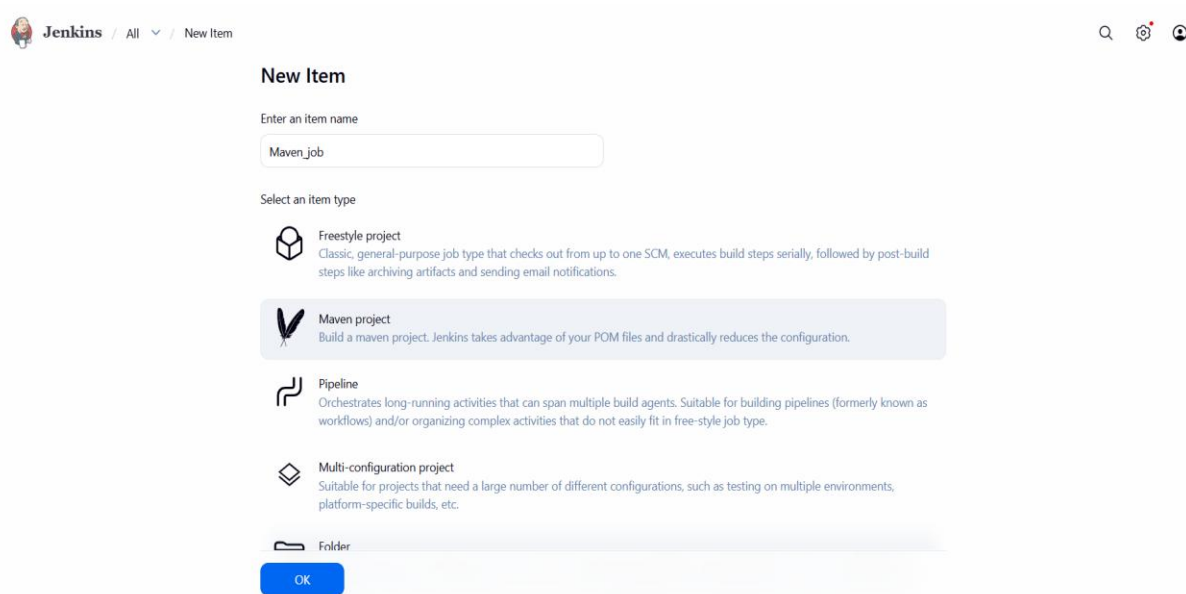


Create one Jenkins job using Maven Project for the code below with two stages:

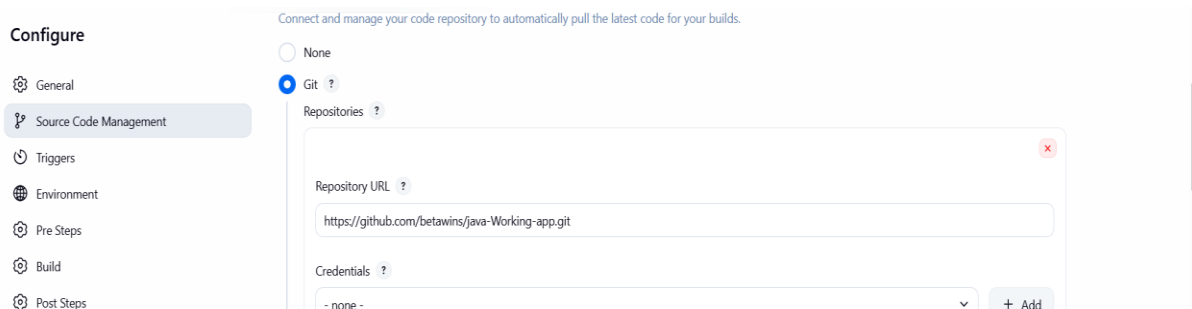
1. Stage 1: Git clone

Stage 2: Maven Compilation Code: <https://github.com/betawins/java-Working-app.git>

- Go to new job or add new item on Jenkins dashboard
- Provide a name and select Maven job and click ok



- Go to source code, and select Git and provide URL of git repo



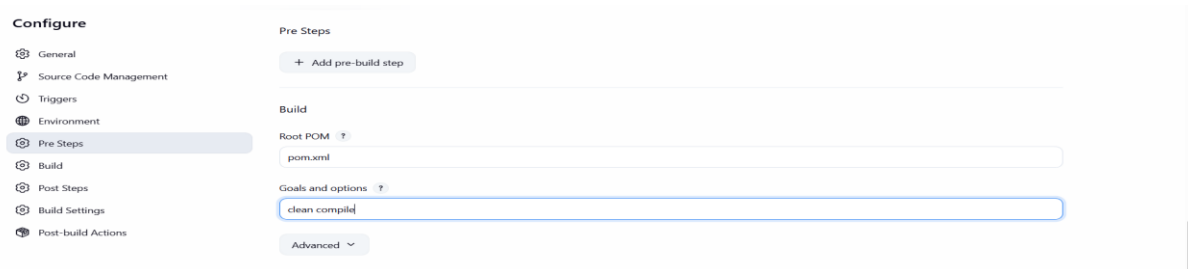
- select Branch name, provide exact branch which is there in repo.



1. Stage 1: Git clone

Stage 2: Maven Compilation Code: <https://github.com/betawins/java-Working-app.git>

- Go to build step and in 'ROOT POM' provide name pom.xml
- Add Goal as 'clean compile'



- After adding build step save job
- Build the job and it will be executed

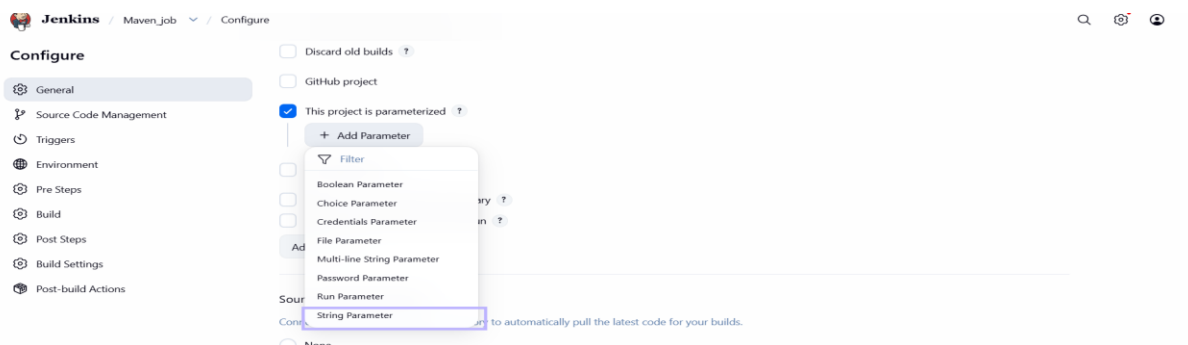


Use the same code and create a parameterized job in Jenkins with:

1. Stage 1: Git clone

Stage 2: Maven Compilation Code: <https://github.com/betawins/java-Working-app.git>

- Go to Maven_job which we created for earlier task and click on configure
- Now select 'This job is parameterised' and select option as string parameter



- Provide a name to the string parameter
- Give default value as branch name

The screenshot shows the Jenkins 'Configure' page for a project. On the left, a sidebar lists various configuration sections: General, Source Code Management, Triggers, Environment, Pre Steps, Build, Post Steps, Build Settings, and Post-build Actions. The 'General' section is selected. A blue banner at the top indicates 'This project is parameterized'. Below this, a 'String Parameter' configuration box is shown with the following fields: 'Name' (GIT_BRANCH), 'Default Value' (main), and 'Description' (Git branch to build).

- Go to git checkout section
- Select source code as URL
- In branch provide Branch name as */\$GIT_BRANCH(just refer the branch which we provided in above parameter)

The screenshot shows the Jenkins 'Configure' page for a project, specifically the 'Source Code Management' section. The 'Branches to build' field is highlighted with a purple box. The 'Branch Specifier (blank for 'any')' field contains the value '*/\$GIT_BRANCH'. Other fields visible include 'Source Code Management' (set to 'none') and 'Add Repository'.

- In build step (Maven section) select ROOT PDM as pom.xml
- In Goals type 'clean compile'

The screenshot shows the Jenkins 'Configure' page for a project, specifically the 'Build' section. The 'Root POM' field is set to 'pom.xml' and the 'Goals and options' field is set to 'clean compile'. The 'Advanced' dropdown is visible below these fields.

- Save the job and then click on build

The screenshot shows the Jenkins 'Maven_job' page. On the left, a sidebar lists various actions: Status, Changes, Workspace, Build with Parameters (highlighted with a purple box), Configure, Delete Maven project, Modules, and Rename. On the right, the 'Maven_job' status is shown as 'Completed'. Below this, the 'Permalinks' section displays a list of recent builds: 'Last build (#1), 18 min ago', 'Last stable build (#1), 18 min ago', 'Last successful build (#1), 18 min ago', and 'Last completed build (#1), 18 min ago'. An 'Add description' button is also visible.

➤ After building job will successfully built as we can see in the image

The image consists of two screenshots of the Jenkins web interface, demonstrating the configuration and execution of a Maven job.

Top Screenshot: Maven project Maven_job

- Header:** Jenkins / Maven_job
- Left Sidebar:** Status, Changes, Workspace, Build with Parameters, Configure, Delete Maven project, Modules, Rename.
- Main Content:**
 - Maven project Maven_job**
 - This build requires parameters:**
 - GIT_BRANCH**
Git branch to build
 -
 - Buttons:** Build (green), Cancel (grey)

Bottom Screenshot: Parameters

- Header:** Jenkins / Maven_job / #2 / Parameters
- Left Sidebar:** Status, Changes, Console Output, Edit Build Information, Delete build "#2", Parameters (highlighted).
- Main Content:**
 - Parameters** (with a green checkmark icon)
 - GIT_BRANCH**
Git branch to build
 -